

模块二：Agent 版本管理 (Agent Version Management)

接口说明

该接口用于维护被测 Agent 的版本信息。支持注册新的 Agent 版本、更新配置参数、以及管理版本的生命周期状态。

1. Agent 版本 (Agent Version) 实体字段说明

参数名	类型	是否必填	说明
id	String (UUID)	自动生成	版本的唯一标识符
name	String	是	版本名称（唯一），如 <code>FactChecker-V1-ReAct</code>
description	String	否	该版本的详细描述或实验背景
status	String	是	状态枚举： <code>ACTIVE</code> (可用), <code>INACTIVE</code> (禁用), <code>ARCHIVED</code> (归档)
config	JSON (String)	是	核心配置： 含 API 地址、模型参数、提示词版本等
metadata	JSON (String)	否	扩展信息：如开发者联系人、所属项目组等
usage_count	Integer	系统维护	该版本被评测任务引用的次数统计

基础配置

- 1. 响应格式：所有接口统一返回 `Result<V0>` 对象。
- 2. Host: `localhost:8080`
- 3. 路径前缀: `/api/agents`

接口列表

接口名称	接口描述	Http方法	URL
createAgentVersion	注册新的 Agent 版本	POST	<code>/api/agents</code>
getAgentList	获取 Agent 版本列表	GET	<code>/api/agents</code>
getAgentById	获取单个版本详情	GET	<code>/api/agents/{id}</code>
updateAgent	更新版本信息或配置	PUT	<code>/api/agents/{id}</code>
changeStatus	变更版本状态（启用/禁用/归档）	PATCH	<code>/api/agents/{id}/status</code>
deleteAgent	删除版本（仅限未被任务引用的）	DELETE	<code>/api/agents/{id}</code>

关键接口请求示例

1. 注册 Agent 版本 (createAgentVersion)

POST `localhost:8080/api/agents`

说明: `config` 字段非常灵活。可以把 URL 和特定的模型参数存在这里。

Request Body:

代码块

```
1  {
2    "name": "FactCheck-Sonnet-V1",
3    "description": "使用 Claude opus 4.7 模型的 ReAct 架构核查器",
4    "status": "ACTIVE",
5    "config": {
6      "api_endpoint": "http://agent-service:8000/api/v1/verify",
7      "model_name": "claude-opus-4.7",
8      "temperature": 0,
9      "max_steps": 6,
10     "reflexion_enabled": false
11   },
12   "metadata": {
13     "team": "Alpha-Agent-Dev",
14     "tags": ["ReAct", "High-Precision"]
15   }
16 }
```

Response:

代码块

```
1  {
2    "code": 200,
3    "data": {
4      "id": "550e8400-e29b-41d4-a716-446655440000",
5      "name": "FactCheck-Sonnet-V1",
6      "created_at": "2024-04-08T14:00:00"
7    },
8    "msg": "Agent 版本注册成功"
9  }
```

2. 更新 Agent 配置 (updateAgent)

PUT `localhost:8080/api/agents/{id}`

说明：当 Agent 团队修改了他们的 API 地址或想测试不同的参数时，通过此接口修改。

Request Body:

```
1  {
2      "description": "更新：开启了反思机制的版本",
3      "config": {
4          "api_endpoint": "http://agent-service:8000/api/v1/verify",
5          "model_name": "claude-3-5-sonnet",
6          "temperature": 0.2,
7          "reflexion_enabled": true
8      }
9  }
```

Response:

代码块

```
1  {
2      "code": 200,
3      "data": {
4          "id": "550e8400-e29b-41d4-a716-446655440000",
5          "updated_at": "2024-04-09T09:00:00"
6      },
7      "msg": "配置已更新"
8  }
```

3. 获取 Agent 列表 (getAgentList)

GET `localhost:8080/api/agents?status=ACTIVE`

Response:

代码块

```
1  {
2      "code": 200,
3      "data": [
4          {
5              "id": "550e8400-e29b-41d4-a716-446655440000",
6              "name": "FactCheck-Sonnet-V1",
7              "status": "ACTIVE",
8              "usage_count": 12
9          },
10         {
```

```
11         "id": "661f9511-f30c-52e5-b827-557766551111",
12         "name": "FactCheck-GLM4-Baseline",
13         "status": "ACTIVE",
14         "usage_count": 5
15     }
16 ],
17     "msg": null
18 }
```

设计说明：

1. 版本不可变性提示：

在实际评测平台中，通常建议：如果 Agent 的核心逻辑发生了重大变化，应该**创建一个新版本**，而不是修改旧版本的配置。这样可以确保历史评测结果（Eval Tasks）是有据可查的。

2. 为什么使用 UUID (`id`)? :

正如你设计的字段原因所述，使用 UUID 可以避免在不同环境（开发、测试、生产）迁移数据时发生 ID 冲突，同时也增加了接口安全性（别人猜不到下一个 Agent 的 ID）。

3. Status 的作用：

`ARCHIVED`（归档）状态非常重要。当你不再需要某个旧版 Agent 时，不应直接删除它，否则之前的评测报告会因为找不到关联的 Agent 配置而无法显示详情。将其设为 `ARCHIVED` 可以让它在选择列表里消失，但保留历史数据。

4. Config 的读取：

当后端执行评测任务时，会根据 `task` 关联的 `agent_id` 取出这个 `config` JSON。后端代码会读取其中的 `api_endpoint` 字段，从而动态地知道该把测试数据发往哪个地址。